Enrollment No.: 240840116014

Practical-7

Aim: Develop advanced visualizations (Box Plot, Heatmap, Pair Plot, Violin Plot).

Step 1: Create and inspect the Dataset

```
import numpy as np
#import the pandas library.
import pandas as pd

#imoprt the mutplotlib library for creating graph
import matplotlib.pyplot as plt
import seaborn as sns
```

```
data={"Student_Id":[1,2,3,4,5,6,7,8,9,10,11,12,13,14],
      "Study_Hours":[1.5,2.0,2.5,3.0,3.5,4.0,4.5,5.0,5.5,6.0,6.5,7.0,8.0,9.0],
      "Attendence":[65,70,72,75,78,80,82,85,87,90,91,93,95,97],
      "Assignment_Score":[55,60,62,65,68,72,75,78,80,84,87,90,94,98],
      "Midterm_Marks":[48,52,55,58,61,65,68,72,75,79,82,86,91,96],
      "Final_Marks":[45,50,52,57,62,68,72,76,81,85,89,93,97,100]
}

df = pd.DataFrame(data)
df
```

| | Student_Id | Study_Hours | Attendence | Assignment_Score | Midterm_Marks | Final_Marks |
|---|---|---|---|---|---|---|
| **0** | 1 | 1.5 | 65 | 55 | 48 | 45 |
| **1** | 2 | 2.0 | 70 | 60 | 52 | 50 |
| **2** | 3 | 2.5 | 72 | 62 | 55 | 52 |
| **3** | 4 | 3.0 | 75 | 65 | 58 | 57 |
| **4** | 5 | 3.5 | 78 | 68 | 61 | 62 |
| **5** | 6 | 4.0 | 80 | 72 | 65 | 68 |
| **6** | 7 | 4.5 | 82 | 75 | 68 | 72 |
| **7** | 8 | 5.0 | 85 | 78 | 72 | 76 |
| **8** | 9 | 5.5 | 87 | 80 | 75 | 81 |
| **9** | 10 | 6.0 | 90 | 84 | 79 | 85 |
| **10** | 11 | 6.5 | 91 | 87 | 82 | 89 |
| **11** | 12 | 7.0 | 93 | 90 | 86 | 93 |
| **12** | 13 | 8.0 | 95 | 94 | 91 | 97 |
| **13** | 14 | 9.0 | 97 | 98 | 96 | 100 |

Step 2: create a box plot

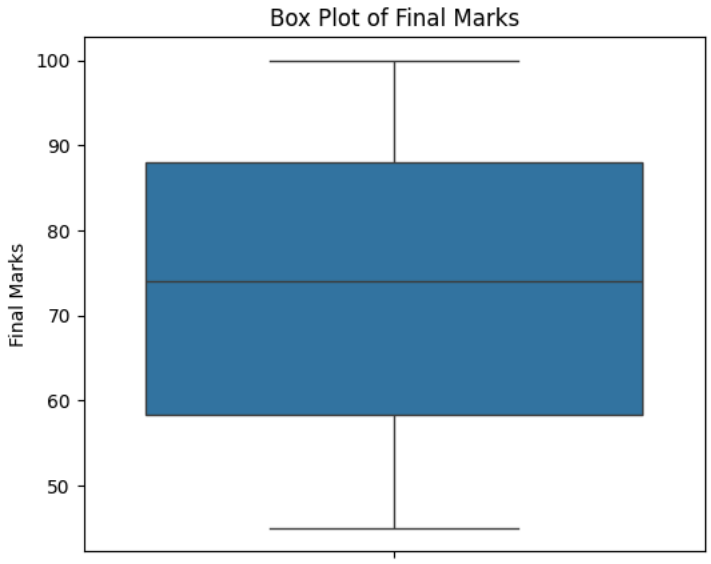
```
# Create a figure with specified size
plt.figure(figsize=(6, 5))

# Create a box plot for Final Marks
sns.boxplot(y=df["Final_Marks"])

# Add title to the plot
plt.title("Box Plot of Final Marks")

# Add label to Y-axis
plt.ylabel("Final Marks")

# Display the plot
plt.show()
```




```
# Select numerical columns for box plot
numerical_columns = [
    "Study_Hours",
    "Attendence",
    "Assignment_Score",
    "Midterm_Marks",
    "Final_Marks"
]

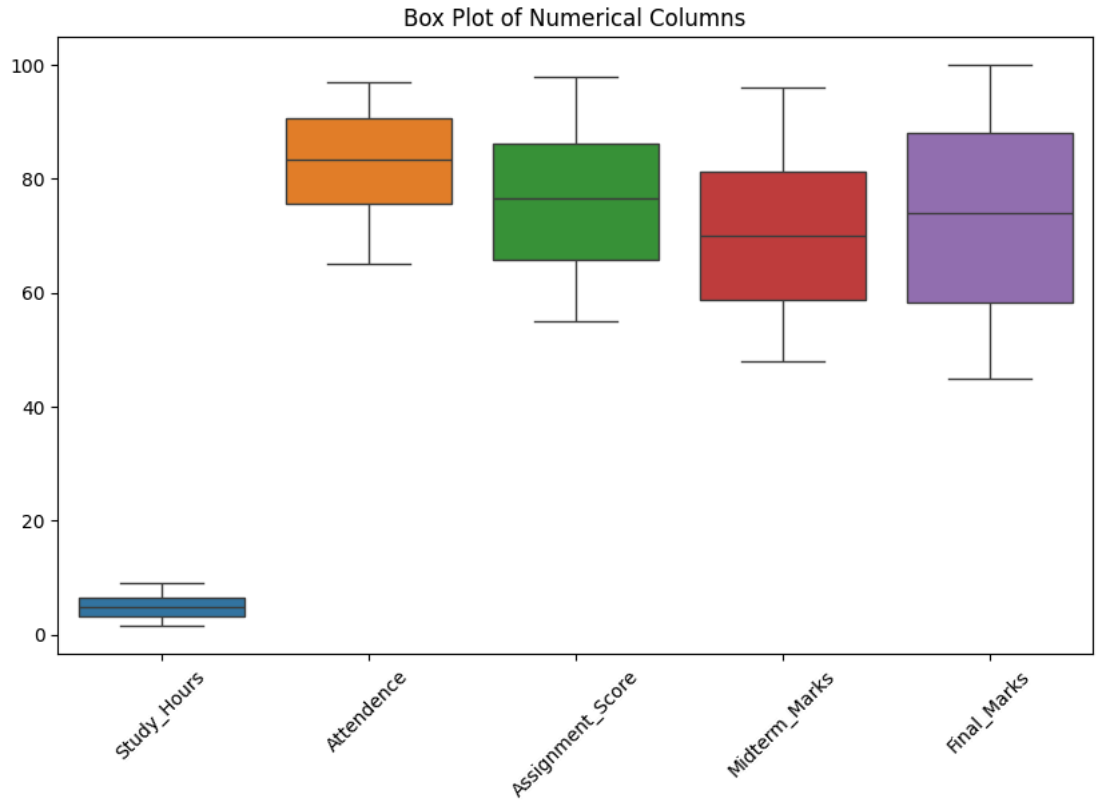
# Set the figure size
plt.figure(figsize=(10, 6))

# Create box plots for all selected numerical columns
sns.boxplot(data=df[numerical_columns])

# Add title
plt.title("Box Plot of Numerical Columns")

# Rotate X-axis labels for better readability
plt.xticks(rotation=45)

# Display the plot
plt.show()
```



Step 3: create a heatmap

```
# Select numerical columns for correlation analysis
numerical_columns = [
    "Study_Hours",
    "Attendence",
    "Assignment_Score",
    "Midterm_Marks",
    "Final_Marks"
]

# Create a new DataFrame containing only numerical columns
numeric_df = df[numerical_columns]
```

```
# Calculate the correlation matrix
correlation_matrix = numeric_df.corr()

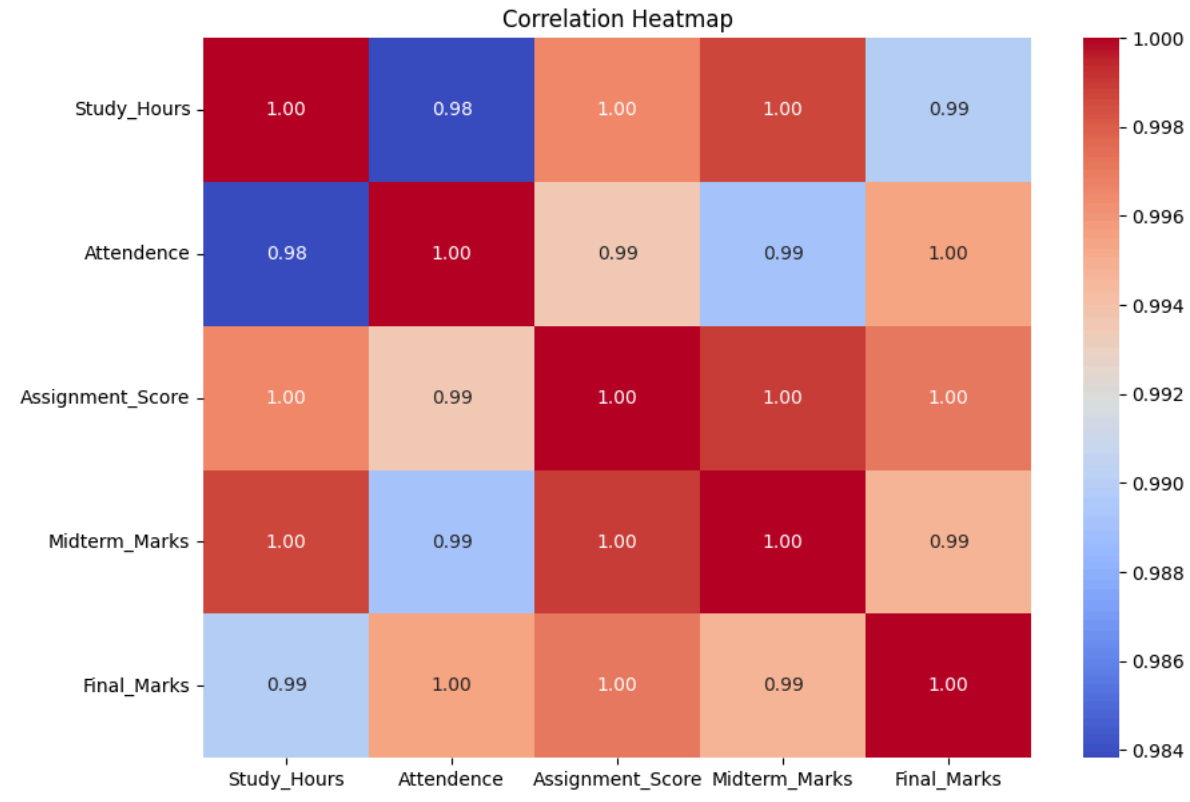
# Display the correlation matrix
correlation_matrix

# Set the figure size
plt.figure(figsize=(10, 7))

# Create the heatmap
sns.heatmap(
    correlation_matrix,
    annot=True,       # Display correlation values inside the cells
    cmap="coolwarm",  # Set the color scheme
    fmt=".2f"         # Display values up to 2 decimal places
)

# Add title to the heatmap
plt.title("Correlation Heatmap")

# Display the heatmap
plt.show()
```




```
# Select numerical columns for correlation analysis
numerical_columns = [
    "Study_Hours",
    "Attendence",
    "Assignment_Score",
    "Midterm_Marks",
    "Final_Marks"
]

# Select only the numerical columns
```

```
numeric_df = df[numerical_columns]

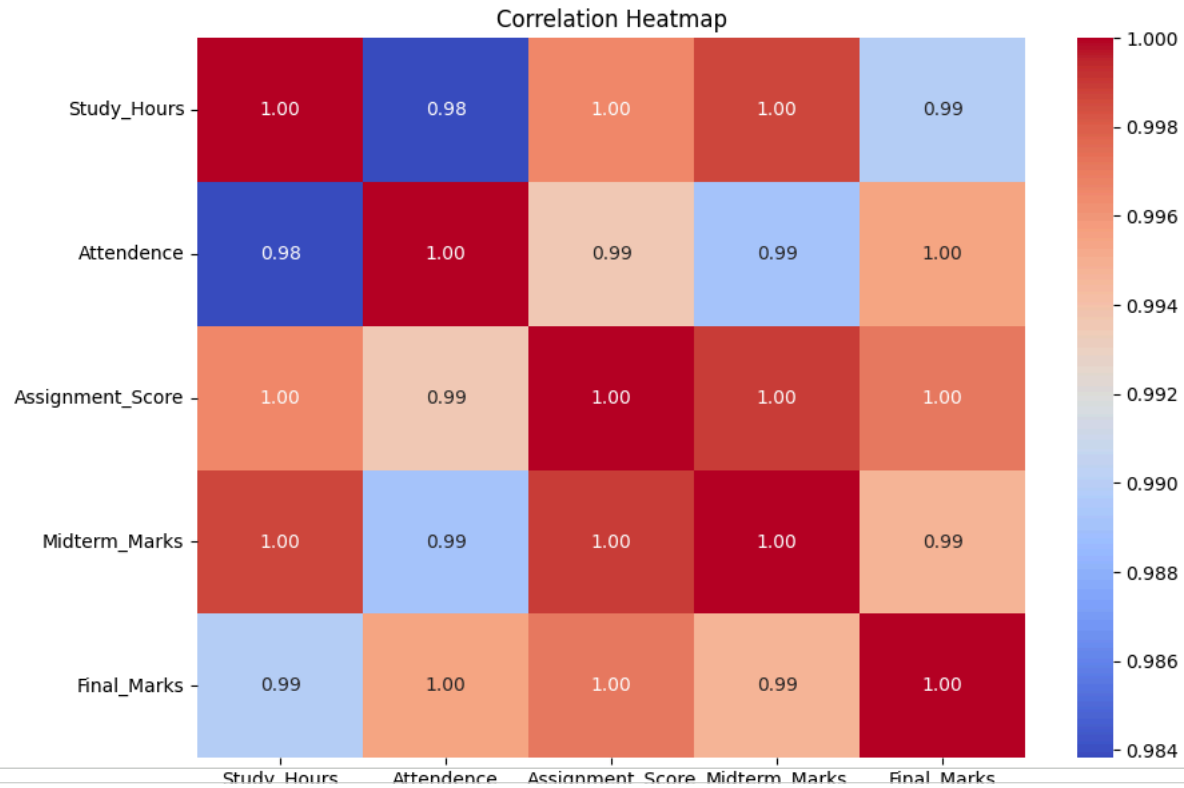
# Calculate correlation matrix
correlation_matrix = numeric_df.corr()

# Set the figure size
plt.figure(figsize=(10, 7))

# Create correlation heatmap
sns.heatmap(
    correlation_matrix,
    annot=True,       # Display correlation values
    cmap="coolwarm",  # Use coolwarm color palette
    fmt=".2f"         # Display 2 decimal places
)

# Add title
plt.title("Correlation Heatmap")

# Display the heatmap
plt.show()
```




```
# Select numerical columns
numerical_columns = [
    "Study_Hours",
    "Attendence",
    "Assignment_Score",
    "Midterm_Marks",
    "Final_Marks"
]

# Calculate the correlation matrix
correlation_matrix = df[numerical_columns].corr()

# Display the correlation matrix
print("Correlation Matrix:")
print(correlation_matrix)
```

```
Correlation Matrix:
                 Study_Hours  Attendence  Assignment_Score  Midterm_Marks  \
Study_Hours         1.000000    0.983822          0.996554       0.998703
Attendence          0.983822    1.000000          0.993531       0.989171
Assignment_Score    0.996554    0.993531          1.000000       0.998949
Midterm_Marks       0.998703    0.989171          0.998949       1.000000
Final_Marks         0.989873    0.995492          0.997041       0.994785

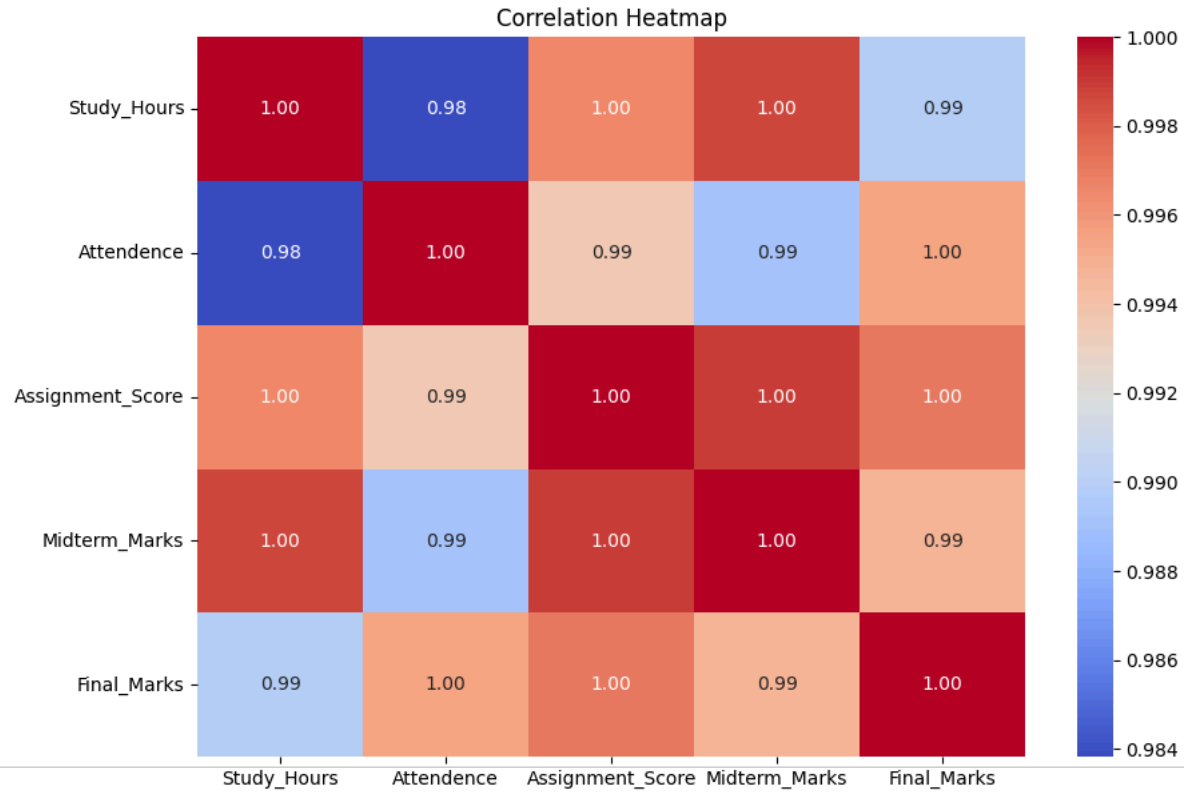
                 Final_Marks
Study_Hours         0.989873
Attendence          0.995492
Assignment_Score    0.997041
Midterm_Marks       0.994785
Final_Marks         1.000000
```

```
# Set the figure size
plt.figure(figsize=(10, 7))

# Create heatmap using the correlation matrix
sns.heatmap(
    correlation_matrix,
    annot=True,       # Display correlation values
    cmap="coolwarm",  # Set color palette
    fmt=".2f"         # Display values up to 2 decimal places
)

# Add title
plt.title("Correlation Heatmap")

# Display the heatmap
plt.show()
```

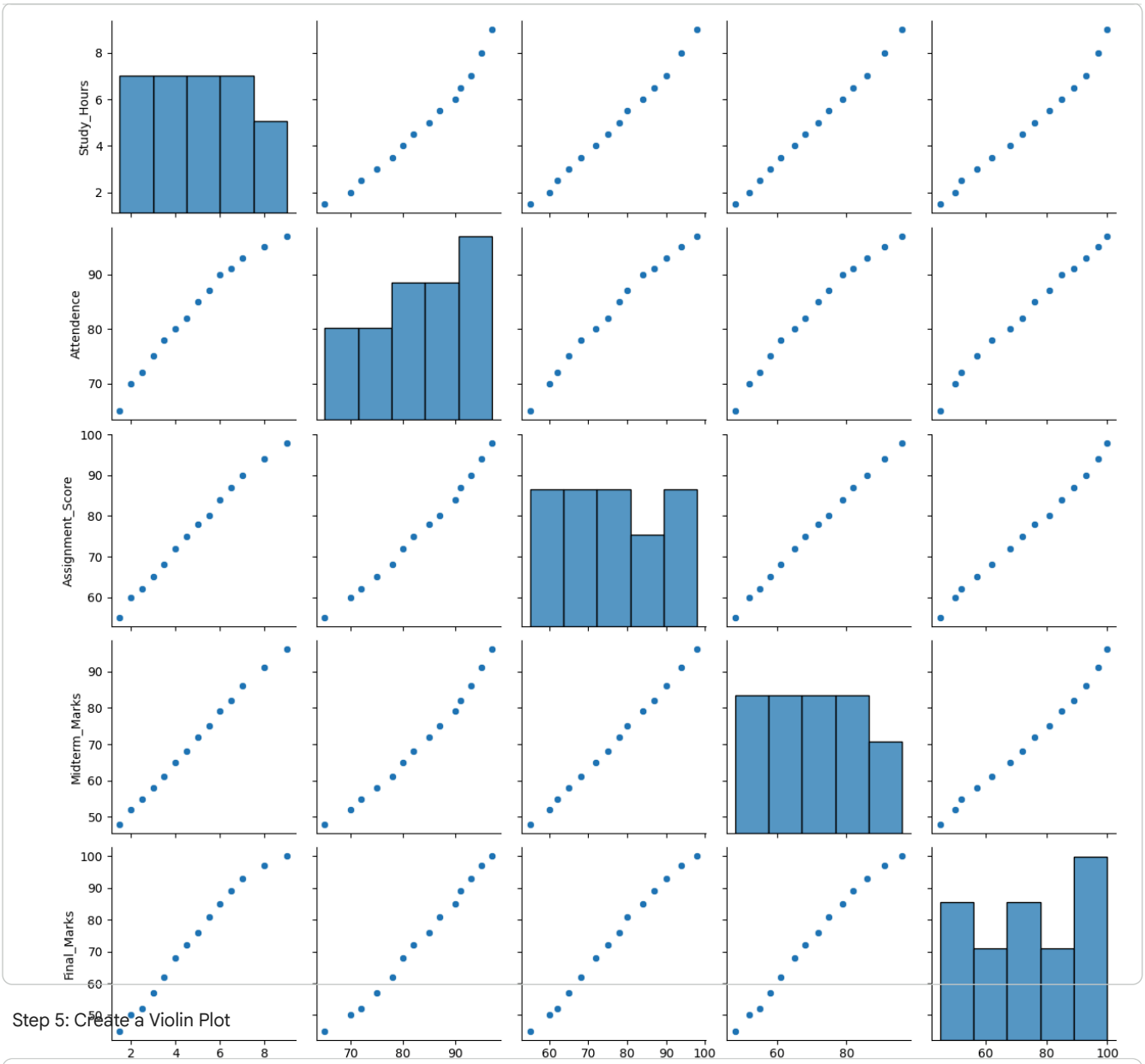



Step 4: Create a Pair Plot

```
# Select numerical columns for comparison
numerical_columns = [
    "Study_Hours",
    "Attendence",
    "Assignment_Score",
    "Midterm_Marks",
    "Final_Marks"
]

# Create a pair plot
sns.pairplot(df[numerical_columns])

# Display the plot
plt.show()
```




Step 5: Create a Violin Plot

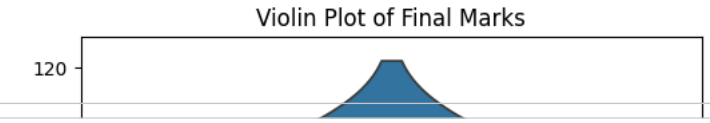
```
# Set the figure size
plt.figure(figsize=(6, 5))

# Create a violin plot for Final Marks
sns.violinplot(y=df["Final_Marks"])

# Add title
plt.title("Violin Plot of Final Marks")

# Add Y-axis label
plt.ylabel("Final Marks")

# Display the plot
plt.show()
```




```
# Select Assignment, Midterm and Final marks
marks_columns = [
    "Assignment_Score",
    "Midterm_Marks",
    "Final_Marks"
]

# Set the figure size
plt.figure(figsize=(9, 6))

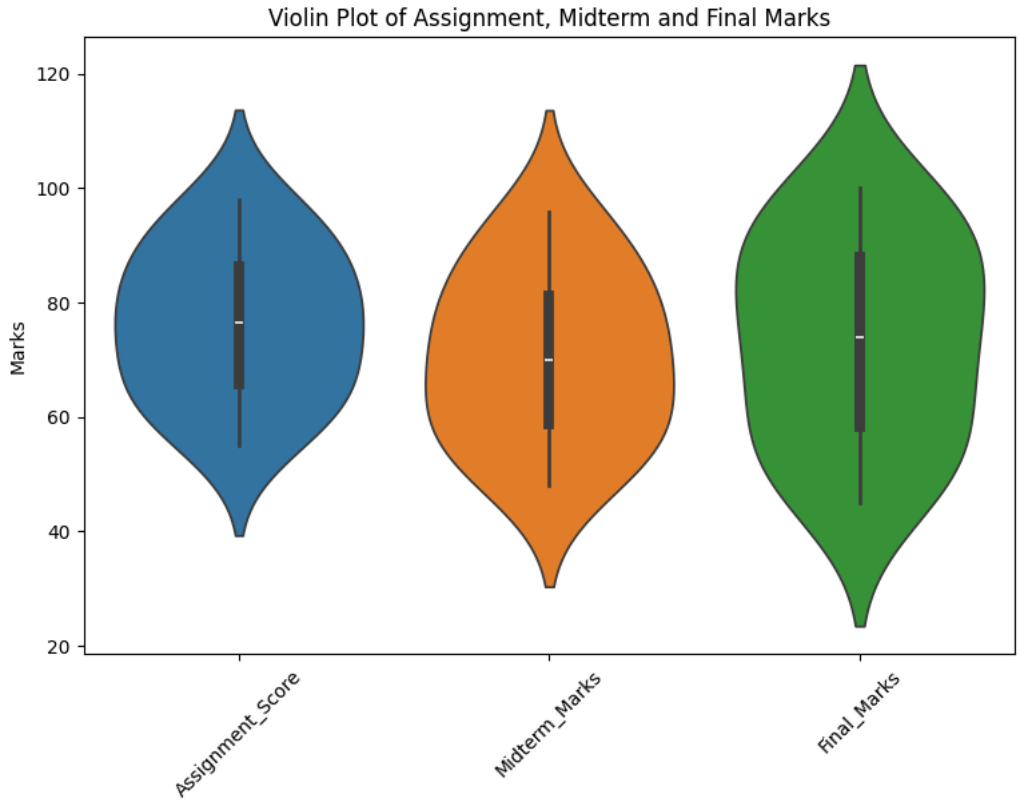
# Create violin plots for selected columns
sns.violinplot(data=df[marks_columns])

# Add title
plt.title("Violin Plot of Assignment, Midterm and Final Marks")

# Rotate X-axis labels for better readability
plt.xticks(rotation=45)

# Add Y-axis label
plt.ylabel("Marks")

# Display the plot
plt.show()
```




Step 6: Save the Dataset as a CSV File

```
# Save the student performance dataset as a CSV file
df.to_csv("student_performance.csv", index=False)

# Import Google Colab files module
from google.colab import files

# Download the CSV file
files.download("student_performance.csv")
```